XENOVERIFY

Complete Codebase Layman Guide, Operational System Walkthrough, and 40-Minute Video Defense Master Blueprint

Academic Research Prototype | Degree: Master of Research (MRe) | Standard: NIST FIPS 204 ML-DSA-65 & Salted Merkle Tree Engine

Project Element	Specification / Empirical Value	Academic Context
Artefact Name	Xenoverify (Post-Quantum Verifiable Digital Identity)	DSRM Activity 3 & 4 (Peffers et al., 2007)
Target Degree	Master of Research (MRe) in Cybersecurity / Computer Science	UK Academic Framework & Peer Governance
PQC Signature Standard	NIST FIPS 204 ML-DSA-65 (Module-Lattice Dilithium3)	Category 3 Security (AES-192 equivalent margin)
Standardised Signature Size	3,309 Bytes (Standard wire format with 16B domain context wrapper)	Constant size regardless of attribute count
Selective Disclosure Primitive	Salted Binary Merkle Tree (128-bit CSPRNG Salt per attribute)	SHA-256 Collision Resistance & Grover Preimage Safe
Verification Latency	80.09 ms Mean (Median 66.25 ms, p95 136.35 ms) in pure Python bytecode	Imperceptible to human operators (<100 ms)
Negative Attack Rejection	100.0% Rejection across 7 adversarial vectors (350 automated tests)	Zero attribute forgery, splicing, or linkage

1. The Layman's Intuition: Cryptography Explained Simply

Digital identity systems today stand on the brink of an unprecedented existential crisis. To explain this clearly to anyone—whether a supervisor, a peer, or a general audience—we break down the core concepts into four intuitive analogies.

1.1 The Quantum Problem: Why Today's Encryption Will Collapse

Every time you log into your bank, verify a digital passport, or submit a signed transaction, your security depends on **Public Key Cryptography**—specifically schemes like RSA or Elliptic Curve Cryptography (ECDSA). These classical schemes rely on math problems that are easy to compute in one direction, but take ordinary supercomputers billions of years to reverse (factoring giant numbers or finding discrete logarithms).

The Shor's Algorithm Threat: In 1994, mathematician Peter Shor proved that if a machine with quantum properties (superposition and entanglement) is ever built, it can solve both factoring and discrete logarithms in minutes. When a Cryptanalytically Relevant Quantum Computer (CRQC) becomes operational, **every classical digital signature (ECDSA, RSA) becomes completely forgeable**.

The 'Harvest-Now-Decrypt-Later' (HNDL) Reality: Attackers are not waiting for quantum computers to exist. Hostile adversaries are intercepting and archiving sensitive digital identity transactions *right now*, knowing that within the next 5 to 10 years, quantum processors will allow them to decrypt and forge identity credentials retroactively.

1.2 The Post-Quantum Solution: What is a Lattice?

How do we build math that even a quantum computer cannot solve? We use **Lattice-Based Cryptography**.

LAYMAN ANALOGY: LATTICE CRYPTOGRAPHY

The Multidimensional Grid Analogy: Imagine a sheet of graph paper. In 2 dimensions, finding a point is trivial. Now imagine that graph paper existing in **500 dimensions** with millions of intersecting grid points (a lattice). The post-quantum engine picks one specific grid point, adds a tiny bit of random error (a slight 'wobble' in all 500 directions), and asks an attacker to find the exact nearest grid point. Without the secret mathematical key, even quantum computers testing in parallel get lost in the geometric maze. This is the **Module Learning With Errors (M-LWE)** problem standardised in **NIST FIPS 204 ML-DSA-65**.

1.3 The Privacy Dilemma & Selective Disclosure

When the government signs a traditional driver's license or passport, they sign the *entire document as a single blob*. If you visit a building and the security guard only needs to verify that you are an adult, you are forced to show your entire ID: revealing your home address, National Identity Number (NIN), date of birth, and clearance level. This gross over-sharing violates the **General Data Protection Regulation (GDPR Article 5(1)(c) - Data Minimisation)**.

Selective Disclosure is the superpower that allows you to prove to a verifier: *'I am a Nigerian citizen and my name is Olushola James'*, while keeping your NIN, exact birthdate, and home address 100% hidden and mathematically un-guessable, yet provably authentic!

1.4 The Salted Binary Merkle Tree: Why Salting is Mandatory

A **Hash Function (SHA-256)** is a mathematical one-way blender: you put in data, and it produces a unique 64-character hexadecimal fingerprint. You cannot un-blend the fingerprint to recover the original data.

THE SALTED ENVELOPE ANALOGY

Why Salt is Critical: Suppose your ID reveals *'nationality: Nigerian'*. If an attacker knows there are only 195 countries in the world, they can hash all 195 countries in 1 millisecond. If hash #42 matches your leaf hash, they just discovered your nationality even though you kept it hidden! To stop this, Xenoverify attaches a fresh **128-bit cryptographically secure pseudorandom salt** (32 random hex characters, e.g. 5c4d3e2f...) to every single attribute. An attacker would have to guess through **2^128 (340 undecillion)** possibilities, which is physically impossible. This provides **Zero Attribute Leakage**.

1.5 The Two-Tiered Envelope: Signing the Root, Not the Plaintext

The primary architectural breakthrough of Xenoverify is how post-quantum signatures interface with selective disclosure:

- **Step 1 (Merkle Tree):** All salted attribute leaves are combined in pairs like a pyramid. The top of the pyramid is the 256-bit **Merkle Root R**.
- **Step 2 (Authenticated Envelope M):** The issuer does NOT sign your personal plaintext. Instead, the issuer wraps the Merkle Root R together with the Credential ID, Issuer DID, Subject DID, and Schema ID into an envelope digest M.
- **Step 3 (Post-Quantum Signature):** The issuer signs ONLY M using ML-DSA-65, generating a single, fixed-size **3,309-byte signature**.

This guarantees that whether you disclose 1 attribute or 100 attributes, the signature size never changes, unrevealed data never leaks, and an attacker cannot splice attributes from someone else's ID!

2. Full Codebase Architecture & File-by-File Breakdown

The Xenoverify codebase is structured into a decoupled Domain-Driven Service-Oriented Architecture (SOA) across backend (Python/Django) and frontend (React/Vite). Below is the complete explanation of every critical file in the repository.

Layer / Module	File Path	Operational Responsibility	Layman Function
Data Persistence	core_identity/models.py	Simulated DID ledger, credentials & audit logs	The system's database memory & simulated blockchain
Cryptographic Engine	core_identity/crypto_service.py	ML-DSA-65 (FIPS 204) & ECDSA secp256k1 implementation	The mathematical engine creating post-quantum keys & signatures
Selective Disclosure	core_identity/merkle_engine.py	128-bit salting, Merkle tree construction & sibling proofs	The privacy engine that hides attributes and proves authenticity
Issuance Domain	core_identity/services/credential_service.py	Coordinates salting, tree synthesis, envelope signing & DB save	The government authority orchestrator creating new ID cards
Verification Domain	core_identity/services/verification_service.py	Executes strict 3-stage validation pipeline	The security gatekeeper verifying proofs, roots, and PQC signatures
Empirical Benchmarks	core_identity/services/benchmark_service.py	Automated N-trial latency and byte dimension profiler	The stopwatch measuring performance of PQC vs classical crypto
API Controller Tier	core_identity/api/views/*.py	8 RESTful HTTP endpoints conforming to OpenAPI/Swagger	The communication bridges between the React frontend and Python engine
Security Test Suite	core_identity/tests.py	Automated negative testing across 7 adversarial attack vectors	The automated hacker simulator proving the system cannot be tricked
Client State Machine	frontend_react/src/App.jsx	React state, tab routing, async API calls & dynamic status feedback	The mission-control brain running in the user's web browser
Client Components	frontend_react/src/components/*.jsx	UI for Scope, Issuer, Holder Wallet, Verifier, Attack Lab & Metrics	The interactive buttons, badges, and forms seen on screen

2.1 Backend Deep Dive: core_identity/

File: core_identity/models.py (The Data Persistence Tier)

This file defines the relational database models using Django's ORM, simulating the on-chain registry without blockchain volatility:

- DIDRegistry: Represents the simulated ledger anchoring decentralized identifiers (e.g. did:xeno:issuer:nimc-01), the entity name, and the 1,952-byte public key.
- IssuedCredential: Stores issued verifiable credentials, containing the 256-bit Merkle root, the 3,309-byte ML-DSA-65 signature, encrypted attribute JSON, and the dynamic boolean is_revoked flag.
- EvaluationLog: Records empirical benchmarks for Chapter 4 (keygen latency, signing latency, verify latency, and storage footprints).

File: core_identity/crypto_service.py (The Pure Cryptographic Engine)

Isolates pure mathematical operations from web abstractions. Contains two primary classes:

- PQCService: Wraps NIST FIPS 204 ML-DSA-65 (Dilithium3). Implements generate_keypair() returning a 1,952-byte public key and 4,032-byte secret key; sign() producing the canonical 3,309-byte wire format; and verify() checking lattice equations.
- ECDSABaselineService: Implements classical ECDSA over curve secp256k1 using OpenSSL C-bindings, generating 88-byte public keys and 71-byte signatures for comparative baseline benchmarking.

File: core_identity/merkle_engine.py (The Selective Disclosure Engine)

The mathematical heart of privacy-preserving disclosure. Contains:

- canonical_attribute_encoding(k, v, s): Formats each claim into deterministic UTF-8 bytes key:value:salt.
- hash_leaf(k, v, s): Computes SHA-256 leaf commitment L_i.

- hash_pair(left, right): Sorts adjacent child hashes lexicographically min || max before hashing to eliminate second-preimage attacks.
- compute_authenticated_envelope_message(...): Builds the domain-separated message M binding the root with issuer and subject DIDs.
- SaltedMerkleTree: Manages odd-leaf duplication, builds binary tree levels, and extracts minimal sibling inclusion paths for revealed claims.

File: core_identity/services/verification_service.py (The 3-Stage Pipeline)

Implements the strict multi-stage verification pipeline required for trusting a credential presentation:

- **Stage 1 (Revocation Accumulator):** Checks the simulated registry to ensure the credential has not been suspended or revoked.
- **Stage 2 (Merkle Proof Inclusion):** Recomputes candidate leaf hashes from revealed claims and salts, climbs sibling authentication paths, and asserts candidate root $R' == R$.
- **Stage 3 (Post-Quantum Signature Verification):** Compiles candidate envelope message M' and validates the 3,309-byte ML-DSA-65 signature σ against the issuer's public key.

File: core_identity/tests.py (The Automated Negative Security Suite)

Contains 350 automated tests executing the 7 negative attack vectors (AV1 to AV7): leaf tampering, path forgery, signature corruption, root substitution, cross-credential claim splicing, revocation bypass, and presentation replay. All 350 tests achieve a 100.0% empirical rejection rate.

2.2 Frontend Deep Dive: frontend_react/

The frontend is built using React 18 and Vite, delivering real-time cryptographic feedback with zero page reloads:

- src/config.js: Automatically detects whether you are running locally (localhost:5173 -> routes to http://localhost:8000/api) or in production.
- src/App.jsx: Central state machine managing active tab, loaded credentials, selected claims, live latency telemetry, and asynchronous loading spinners.
- src/components/ScopeTab.jsx: Renders the DSRM Implementation Scope Matrix (Table 3.1 & 4.1), visually demonstrating implemented vs simulated boundaries.
- src/components/IssuerTab.jsx: UI for generating authority keypairs, entering synthetic applicant claims, and triggering ML-DSA-65 credential issuance.
- src/components/DisclosureTab.jsx: Contains the Holder Wallet (with interactive eye toggle pills for revealing/hiding attributes), the Verifier Panel (displaying emerald verified status), and the 1-Click Attack Simulator.
- src/components/BenchmarkTab.jsx: Renders parametric benchmark controls (10 to 50 iterations), generating real-time comparative charts comparing ML-DSA-65 against ECDSA.

3. The Live System in Action: Step-by-Step Operational Guide

This section provides the exact walkthrough to follow when demonstrating the live system on screen or during evaluation.

& Tab	User Interaction & On-Screen Click	Backend Cryptographic Operation	Visual Feedback & UI State	
Environment metry	Open browser at http://localhost:5173/ and observe the top navigation bar.	Calls GET /system/info/ fetching cryptographic algorithms, library versions, and degree metadata.	Header displays 'NIST FIPS 204 ML-DSA-65', 'MRe DSRM Framework', and active green backend status.	
DSRM Scope	Click on the 'DSRM Scope' tab (first tab on left).	Renders Table 4.1 Implementation Scope and Boundary Matrix directly from research design.	Explains clear demarcation: crypto & proofs are IMPLEMENTED; ledger & wallet are SIMULATED.	
Authority s	Navigate to 'Issuer Authority' tab. Click 'Re-Generate Keypair'.	Calls POST /issuers/initialize/. Executes Dilithium3.keygen(), anchoring DID to database.	Displays animated progress: 'Generating NIST FIPS 204 keypair...'. Outputs 1,952-byte public key.	
Credential ce	Enter synthetic claims (Olushola James, NIN: 29837492812). Click 'Issue and Sign (ML-DSA-65)'.	Calls POST /credentials/issue/. Salts claims with 128-bit CSPRNG, builds Merkle tree, signs envelope M.	Shows 3,309-byte post-quantum signature, 256-bit Merkle root, and saves credential to wallet.	
Selective sure	Navigate to 'Selective Disclosure' tab. In Holder Wallet, toggle 'fullName' & 'nationality' to REVEALED.	Leave 'nin', 'dob', 'residence' marked HIDDEN. Click 'Generate Merkle Proof'.	Calls POST /proofs/generate/. Extracts minimal sibling authentication hashes omitting hidden plaintext.	Wallet displays generated pro packet: 2 revealed claims, 4 h claims, proof extraction time ~
Positive ation	In the Verifier Panel on the right, click 'Verify Proof'.	Calls POST /proofs/verify/. Executes 3-stage pipeline: checks revocation, climbs tree, validates ML-DSA-65.	Instantly renders emerald green banner: 'CRYPTOGRAPHICALLY VALID & VERIFIED' in ~80 ms.	
Negative Demo	Click the red 'Simulate Tamper Attack' button, then click 'Verify Proof'.	Injects a 1-byte alteration into the revealed name ('Olushola James' -> 'Olushola Jamez').	Leaf hash diverges, causing Merkle root mismatch. System catches it with 100% rejection in Stage 2.	
Empirical marks	Navigate to 'Benchmarks & Metrics' tab. Select 20 iterations, click 'Run Evaluation Benchmark'.	Calls POST /benchmarks/run/. Sequentially executes ML-DSA-65 vs ECDSA keygen, signing, and verifying.	Renders comparative bar charts: highlights 3,309 B vs 71 B signature, and 80 ms vs 2 ms verify.	

4. The 40-Minute Video Defense Master Blueprint

This master script is calibrated for a professional, engaging 40-minute academic presentation. It includes exact timestamps, instructions on what to show on screen, and a natural spoken script designed for confidence and academic authority.

TIMECODE	**00:00 - 05:00 — Part 1: The Hook, Context & The Post-Quantum Identity Crisis**
ON-SCREEN ACTION	Face camera with professional introduction. Transition screen to show title slide: 'Xenoverify: Post-Quantum Digital Credentials'. Open browser showing the live platform.
SPOKEN WORDS	*"Hello everyone, and welcome to this technical demonstration and defense of Xenoverify. Today, digital identity systems face a critical dilemma. On one hand, data privacy laws like GDPR and eIDAS 2.0 strictly demand data minimisation—holders should never reveal more information than necessary. On the other hand, the cryptographic foundations of our entire digital society—RSA and Elliptic Curve signatures—are facing complete extinction from Shor's algorithm running on future quantum computers. Hostile entities are already harvesting encrypted identity traffic today to decrypt later. Xenoverify is an academic research prototype built to resolve this challenge. It unifies the newly finalised NIST FIPS 204 ML-DSA-65 post-quantum signature standard with a Salted Binary Merkle Tree selective disclosure engine, ensuring quantum-resistant authenticity with zero attribute leakage."*

TIMECODE	**05:00 - 12:00 — Part 2: Layman Cryptography & Architectural Principles**
ON-SCREEN ACTION	Show conceptual architecture slides: Lattice grid illustration, Salted Merkle Tree pyramid diagram, and the Domain-Separated Envelope formula M.
SPOKEN WORDS	*"Before we look at the code, let's understand the mathematics in simple terms. How does lattice cryptography protect against quantum computers? Instead of relying on prime numbers, ML-DSA-65 relies on the hardness of finding short vectors in a 500-dimensional grid with added noise. Even a quantum computer gets lost in this geometric labyrinth. Now, why couldn't we just use zero-knowledge zk-STARKs? Early in this research, STARKs were evaluated, but generating a STARK proof took several seconds and required hundreds of megabytes of RAM—completely impractical for a smartphone wallet. Instead, we chose a Salted Binary Merkle Tree. By salting every attribute with an independent 128-bit random token, we eliminate dictionary inversion attacks. And crucially, the issuer signs a domain-separated envelope message M that binds the Merkle root, issuer DID, subject DID, and credential ID into exactly ONE constant-sized 3,309-byte post-quantum signature."*

TIMECODE	**12:00 - 24:00 — Part 3: Deep Dive into the Codebase (File-by-File Walkthrough)**
ON-SCREEN ACTION	Switch screen to VS Code. Walk through the directory tree. Open models.py, crypto_service.py, merkle_engine.py, verification_service.py, and tests.py in split panes.
SPOKEN WORDS	*"Let us examine the codebase. The backend is built in Python 3.11 with Django REST Framework, organised into four decoupled tiers. Here in models.py, we have our simulated ledger entities—the DIDRegistry and IssuedCredential models—anchoring DIDs and tracking revocation status. In crypto_service.py, notice our PQCService class. It interfaces directly with Dilithium3, implementing NIST FIPS 204 key generation and signing, outputting the canonical 3,309-byte wire format. Next, in merkle_engine.py, observe our canonical encoding function. It sorts attributes lexicographically, attaches 128-bit salts, and uses sorted-pair hashing to eliminate second-preimage vulnerabilities. In verification_service.py, you can see our strict three-stage verification pipeline: Stage 1 checks revocation; Stage 2 climbs the Merkle tree to assert R-prime equals R; and Stage 3 validates the ML-DSA-65 signature. Finally, in tests.py, our negative security testing battery runs 350 automated trials across seven attack vectors—achieving an empirical 100% rejection rate."*

TIMECODE	**24:00 - 33:00 — Part 4: Live Interactive Demonstration & Attack Simulation**
ON-SCREEN ACTION	Switch screen to browser at http://localhost:5173/. Walk through Issuer Tab -> Disclosure Tab -> Verifier Panel -> Tamper Attack simulation.

SPOKEN WORDS	*"Now, let's watch Xenoverify in action. In the Issuer Authority tab, I click 'Re-Generate Keypair'. Instantly, the system generates a fresh ML-DSA-65 keypair with a 1,952-byte public key. I enter synthetic claims for Olushola James, including his NIN, birthdate, and clearance level, and click 'Issue and Sign'. The server salts each attribute, builds the Merkle tree, and generates a 3,309-byte signature over the envelope. Now, in the Selective Disclosure tab, I act as the citizen holding this wallet. A verifier only asks to confirm my name and nationality. I toggle fullName and nationality to Revealed, while leaving nin, dob, and clearance hidden. I click 'Generate Merkle Proof'—it takes just 2 milliseconds. On the right, the verifier evaluates the presentation. I click 'Verify Proof'—and in 80 milliseconds, we receive an emerald green 'CRYPTOGRAPHICALLY VALID & VERIFIED' confirmation! Now, to prove robustness: I click 'Simulate Tamper Attack', which alters one byte of my name. I click verify again—and immediately, the pipeline rejects it with a Merkle proof inclusion failure. The integrity cannot be breached."*

TIMECODE	**33:00 - 37:00 — Part 5: Empirical Benchmark Results & The 100-Trial Feasibility Milestone**
ON-SCREEN ACTION	Switch to the Benchmark Tab in the browser. Show the latency charts. Display Table 4.4 and Table 4.5 from Chapter 4 on screen.
SPOKEN WORDS	*"In strict compliance with our DSRM evaluation protocol, we executed a 100-run feasibility milestone on pinned hardware. As shown in Table 4.4, issuance averages 533 milliseconds with a median of 338 milliseconds. Proof extraction on the wallet device averages just 2.18 milliseconds. And relying party verification averages 80.09 milliseconds—well under the 100-millisecond threshold for interactive human response. Comparing ML-DSA-65 against classical ECDSA in Table 4.5: public keys expand by 22 times and signatures expand by 46 times. However, in modern 5G networks, transmitting an extra 3.3 kilobytes consumes less than a millisecond. Crucially, our pure Python implementation runs in interpreted bytecode; native C implementations execute ML-DSA-65 in under 0.3 milliseconds, confirming that post-quantum digital identity is completely feasible today."*

TIMECODE	**37:00 - 40:00 — Part 6: Supervisory Defense, Q&A; Preparation & Conclusion**
ON-SCREEN ACTION	Face camera with summary slide. Conclude confidently with research contributions, boundaries, and future enterprise work.
SPOKEN WORDS	*"To conclude, Xenoverify successfully demonstrates that post-quantum signatures and privacy-preserving selective disclosure can be harmoniously integrated into a functional digital identity pipeline. We have established clear boundaries: the cryptographic engine and verification pipeline are fully implemented in software, while surrounding blockchain ledgers and mobile secure hardware are simulated. This research resolves the identity trilemma, proving that post-quantum digital credentials can be delivered without sacrificing privacy or real-time verification speed. Thank you for your time, and I am now ready to take your questions."*

5. Supervisory & Viva Defense: The 6 Tough Technical Questions

During your master's presentation or oral defense, your supervisor and examiners will test your technical depth. Below are the six most critical questions, along with bulletproof, academically rigorous answers.

Q1: Why did you choose Salted Merkle Trees instead of zk-STARKs for selective disclosure?

zk-STARKs offer general-purpose verifiable computation and zero-knowledge predicate proofs (e.g. proving age >= 18 without revealing birthdate). However, for selective attribute disclosure, STARKs introduce massive proving latency (several seconds) and excessive memory overhead (>250 MB), violating the sub-second requirement for wallet devices. Furthermore, mature STARK libraries are written in C++/Rust without stable Python bindings. In contrast, a Salted Binary Merkle Tree achieves the exact same data minimisation guarantee with sub-5 ms proof extraction, zero trusted setup, and pure collision-resistant hash security (SHA-256), which retains 128-bit security against Grover's quantum search.

Q2: Does your Merkle disclosure mechanism protect against Presentation Linkability?

This is a key architectural trade-off. If a holder presents the identical proof packet (containing the same salts and sibling hashes) to multiple verifiers, those verifiers could collude to correlate the presentations via the shared Merkle Root R. Xenoverify mitigates this at the issuance layer: issuing fresh credentials with fresh 128-bit salts produces completely divergent roots (collision probability ~ 2^-128). True presentation-level unlinkability across repeated presentations of the same credential requires zero-knowledge range proofs, which we have transparently identified in Chapters 3 and 5 as an avenue for future enterprise work.

Q3: Why is ML-DSA-65 signing taking ~338 ms in Python while ECDSA takes only 1.4 ms?

This reflects an underlying software implementation confound rather than a fundamental lattice limitation. Classical ECDSA is executed via Python's cryptography library, which wraps highly optimized, hardware-accelerated OpenSSL C-bindings with dedicated assembly point multiplication. Conversely, ML-DSA-65 is executed via dilithium-py, a pure-Python reference implementation running entirely within interpreted bytecode without SIMD vectorization (AVX2/AVX-512). Published native C implementations (such as liboqs and PQClean) achieve ML-DSA-65 signing in 0.15 to 0.30 ms and verification in 0.08 to 0.15 ms on equivalent x86_64 CPUs.

Q4: Why does the signature size say 3,309 bytes in the dissertation and code?

In August 2024, NIST finalised FIPS 204. The finalised ML-DSA-65 wire format requires exactly 3,309 bytes (comprising response vector z, hint vector h, and context commitment c_tilde). While Round-3 CRYSTALS-Dilithium reference libraries used 3,293 bytes, Xenoverify implements the canonical FIPS 204 serialization wrapper containing the 16-byte domain context trailer, reconciling live prototype telemetry with the finalised NIST standard.

Q5: How is Holder Binding enforced if you don't use a mobile Secure Enclave?

Holder binding is enforced at the cryptographic protocol layer by embedding the Subject DID inside the Domain-Separated Authenticated Envelope Message M signed by the issuer. This cryptographically binds the credential to the subject's identity. However, binding private signing keys to physical hardware (e.g., Apple Secure Enclave or Android StrongBox requiring biometric thumbprint assertion) is intentionally demarcated as simulated in Table 4.1 to maintain focus on software-layer cryptographic integration.

Q6: Why did you simulate the blockchain ledger instead of deploying to Ethereum?

Deploying to a public blockchain introduces external confounding variables such as network latency, gas fee volatility, and consensus delays that distort reproducible cryptographic profiling. Simulating the DID registry using relational database tables provides identical read-lookup semantics with deterministic microsecond latency, allowing the research to isolate and measure pure cryptographic overhead in accordance with DSRM principles.